

Final Exam Adaptation Guide: Rod-Mass to Final System

This guide explains how to quickly adapt the code developed for the `L_rodmass` case study to a new system in the final exam.

1. Directory Setup

If your final exam files are in a new folder (e.g., `src/case_studies/FINAL_SYSTEM/`), you can copy the following core files from `L_rodmass`: `- dynamics.py` - `params.py` - `pid_controller.py` - `ssi_controller.py` - `ssi_dist_obs_controller.py` - `lqr_controller.py` - `eq_controller.py`

2. Adaptation Steps

Step A: Update `params.py`

1. Update the physical constants (m, ℓ, g, b, k_1, k_2 , etc.) to match the final exam's system.
2. Update `tau_max` and simulation parameters (`ts`) if they differ.

Step B: Update `dynamics.py`

1. **The `f(x, u)` method:** This is the most critical change. Update the equation of motion.
 - For a point mass: $\ddot{\theta} = \frac{1}{J}(\sum \text{Torques})$.
 - Ensure you handle the input torque τ correctly (usually `u[0]`).
2. **The `h(self)` method:** Ensure the output matches what the sensors provide (usually just `theta`).

Step C: Update `eq_controller.py`

1. Update the `calculate_ue` method with the new equilibrium derivation:
 - Set $\ddot{\theta} = 0, \dot{\theta} = 0$ in your EOM.
 - Solve for τ_e in terms of θ_e .

Step D: Update Controller Gains

1. **PID (`pid_controller.py`):**
 - Update `self.u_e` in `__init__`.
 - Tuned gains (k_p, k_d, k_i) will likely need adjustment based on the new system bandwidth.
2. **State-Space Controllers:**
 - The A, B, C matrices are automatically calculated in `__init__` based on the linearized model.

- **Crucial:** Ensure the linearized A and B matrices in the `__init__` methods of `ssi_controller.py`, `ssi_dist_obs_controller.py`, and `lqr_controller.py` match your new system's linearization.

3. Fast-Tracking the Simulations

Copy `practiceFinalSim2.py`, `Sim3`, and `Sim4` to your new folder. 1. Update the relative imports: `python from .dynamics import RodMassDynamics as Dynamics` # Change to: `from .dynamics import FinalDynamics as Dynamics` 2. Update references to the system name and parameter values in the print statements.

4. Troubleshooting Checklist

- **IndexError:** Usually means `u` or `input_disturbance` is being treated as a scalar. Use `np.atleast_1d()` or `u[0]`.
- **Inhomogeneous Shape Error:** Ensure `xdot` is a clean numpy array of floats: `np.array([val1, float(val2)])`.
- **Steady-State Error:** If using SSI or PID, ensure `u_e` (feedforward) is correct for the current equilibrium.
- **Instability:** Check if your B matrix in the controller matches the B matrix in the physics. A sign error here will cause the system to fly away.